

Technical Design Document

System architecture, data design, interfaces, security, deployment and operations

Field	Value
Document ID	EXAI-TDD-001
Version	1.1
Status	Released for review
Date	18 September 2026
Owner	Raj Nikam
Classification	Internal

Document Control

Revision History

Version	Date	Author	Description
0.1	14 Sep 2026	Raj Nikam	Initial architecture, data model and knowledge graph design
0.2	15 Sep 2026	Raj Nikam	Session persistence, append-only version history, Excel formula output
0.3	16 Sep 2026	Raj Nikam	PostgreSQL, Neo4j and Redis integration; schema inference for arbitrary files
1.0	17 Sep 2026	Raj Nikam	Clerk authentication, interface redesign, release documentation
1.1	18 Sep 2026	Raj Nikam	Hosted deployment; connection pooling and background graph sync; query isolation, usage limits, security headers and signed snapshots; workspace and landing redesign; legal pages, search metadata and accessibility

Reviewers

Name	Role	Date	Signature

Contents

Document Control.....	2
Revision History	2
Reviewers.....	2
1. Introduction	5
1.1 Purpose	5
1.2 Scope.....	5
1.3 Intended Audience.....	5
1.4 Definitions and Abbreviations.....	5
1.5 References	5
2. System Overview.....	6
2.1 Product Description	6
2.2 Key Capabilities	6
2.3 Design Principles	6

2.4 Assumptions and Constraints	6
3. Architecture	8
3.1 Architectural Style.....	8
3.2 System Architecture.....	8
3.3 Component Responsibilities	8
3.4 Request Flows	9
4. Technology Stack	11
5. Detailed Design	12
5.1 Schema Profiling	12
5.2 Diagnostics	12
5.3 Knowledge Graph.....	12
5.4 Assistant and Tools	13
5.5 Edit Operations	13
5.6 Versioning and Audit.....	13
5.7 Excel Formula Output	13
5.8 Client Application.....	14
5.9 Search, Sharing and Accessibility	14
6. Data Design	15
6.1 Relational Schema.....	15
6.2 Snapshot Storage	15
6.3 Data Ownership	15
7. API Specification.....	16
7.1 Error Responses	16
8. Security	18
8.1 Authentication	18
8.2 Authorisation	18
8.3 Other Controls.....	18
8.4 Usage Limits	18
8.5 Query Isolation.....	18
8.6 Transport and Browser Security	19
8.7 Dependency Audit.....	19
8.8 Residual Risks	19
9. Configuration	20
10. Deployment.....	22
10.1 Environments	22

10.2 Local Setup	22
10.3 Production Deployment.....	22
10.4 Release Procedure	23
10.5 Service Endpoints (Local)	23
11. Performance	25
11.1 Server	25
11.2 Client	25
11.3 Measured Results.....	25
12. Testing.....	26
12.1 Strategy	26
12.2 Test Inventory	26
12.3 Results	26
13. Operations and Maintenance	27
13.1 Health Checks.....	27
13.2 Troubleshooting.....	27
13.3 Extending the System	27
14. Known Limitations and Future Work	29
Appendix A. Repository Structure.....	30
Appendix B. Sample Dataset.....	31

1. Introduction

1.1 Purpose

This document describes the design of ExcelAI, a web application for inspecting, querying and correcting spreadsheet data with the help of a conversational assistant. It records the architecture, component responsibilities, data structures, interfaces, security controls and operating procedures, and serves as the reference for development, review and maintenance.

1.2 Scope

The document covers the web client, the API service, the graph synchronisation worker and the backing services (PostgreSQL, Neo4j, Redis and snapshot storage). It covers the system as built at version 1.1, including the hosted deployment. Items planned but not yet implemented are listed in Section 14.

1.3 Intended Audience

- Software engineers maintaining or extending the system
- Reviewers assessing the design and its trade-offs
- Operators responsible for deployment and support

1.4 Definitions and Abbreviations

Term	Definition
Session	One uploaded workbook together with its versions, audit log and conversation
Version	A complete snapshot of the table. Version 0 is the file as uploaded and is never modified
Audit entry	Record of a change: operation, scope, rows affected, reason, acting user and time
Subject column	The column whose values the rows are mainly about, inferred per file
Flag column	A mostly empty column where a blank means "no" rather than "missing"
GraphRAG	Retrieval of facts from a knowledge graph to ground a language model answer
JWT / JWKS	JSON Web Token / JSON Web Key Set used to verify signed session tokens
LLM	Large language model, accessed through OpenRouter

1.5 References

- IEEE 1016-2009, Standard for Information Technology: Software Design Descriptions
- FastAPI documentation, <https://fastapi.tiangolo.com>
- Neo4j Cypher Manual, <https://neo4j.com/docs/cypher-manual>
- Clerk documentation, <https://clerk.com/docs>
- Project repository: README.md, RUN.md and the docs/ directory

2. System Overview

2.1 Product Description

Users open an Excel workbook in a spreadsheet-style workspace. The application profiles the columns, lists data-quality problems, builds a knowledge graph from the data and provides an assistant that answers questions and applies corrections on request. Every correction creates a new version that can be compared and restored. For each answer and correction, the application also supplies the Excel formula or menu steps that produce the same result in the user's own copy of the file.

2.2 Key Capabilities

- Upload of any .xlsx or .xls file, stored exactly as received
- Automatic inference of column roles, reviewable and editable by the user
- Detection of missing values, disagreeing column pairs and near-duplicate names
- Relationship questions answered from a knowledge graph, with fuzzy name resolution
- Corrections restricted to a fixed set of validated operations
- Append-only version history with cell-level comparison and restore
- Excel formula and procedure output for answers and corrections
- Per-user sign-in and ownership of uploaded data
- Usage limits per user, with exemptions for designated accounts
- Public pages: product landing page, privacy policy, terms of service and a not-found page

2.3 Design Principles

Principle	Application in the system
Data is never cleaned automatically	Uploads are stored unchanged. Detection and correction are separate components; nothing changes without an explicit request.
Reads and writes have different trust levels	Read paths may evaluate a restricted query. The write path accepts only named operations with typed parameters and never runs generated code.
History is append-only	Restoring an earlier version creates a new version, so no audit entry is ever deleted.
Nothing is specific to one dataset	Column roles are inferred from the data; diagnostics and the graph follow those roles.
Claims are verified independently	After a correction, the interface highlights cells from a server-side comparison, not from the assistant's description.

2.4 Assumptions and Constraints

- The first worksheet of a workbook is the data table, with column headers in row 1.
- Files are limited to 25 MB by configuration.
- The assistant requires an OpenRouter API key; all other features work without one.

- Neo4j, PostgreSQL and Redis are optional. Without them the service runs in local mode using SQLite and an in-process graph.
- The hosted deployment does not use Redis; graph updates run on a background thread inside the API process.

Component	Responsibility	Source
Data service	Upload, paged table access, search, sort, comparison and export	<code>main.py</code> , <code>diff.py</code>
Schema profiler	Classifies each column as subject, attribute, measure, date, flag or identifier	<code>schema_profile.py</code>
Diagnostics	Read-only data-quality checks	<code>diagnostics.py</code>
Assistant	Conversation state, tool selection, grounded answers	<code>agent.py</code>
Query sandbox	Runs table queries in a child process with time, CPU and network restrictions	<code>isolated_exec.py</code> , <code>safe_executor.py</code>
Edit service	The only component permitted to change data	<code>safe_editor.py</code>
Session store	Sessions, versions, audit log, conversation; conflict detection; pooled database connections	<code>data_store.py</code>
Snapshot storage	Signed version snapshots on disk or S3-compatible storage	<code>snapshot_storage.py</code>
Graph backend	Retrieval from Neo4j or the in-process graph	<code>graph_backend.py</code> , <code>neo4j_graph.py</code> , <code>graph_builder.py</code>
Graph synchronisation	Applies row changes to Neo4j on a background thread, or through the Redis queue and worker	<code>graph_sync.py</code> , <code>graph_worker.py</code>
Formula generator	Excel formulas and procedures for answers and corrections	<code>excel_formulas.py</code>

3.4 Request Flows

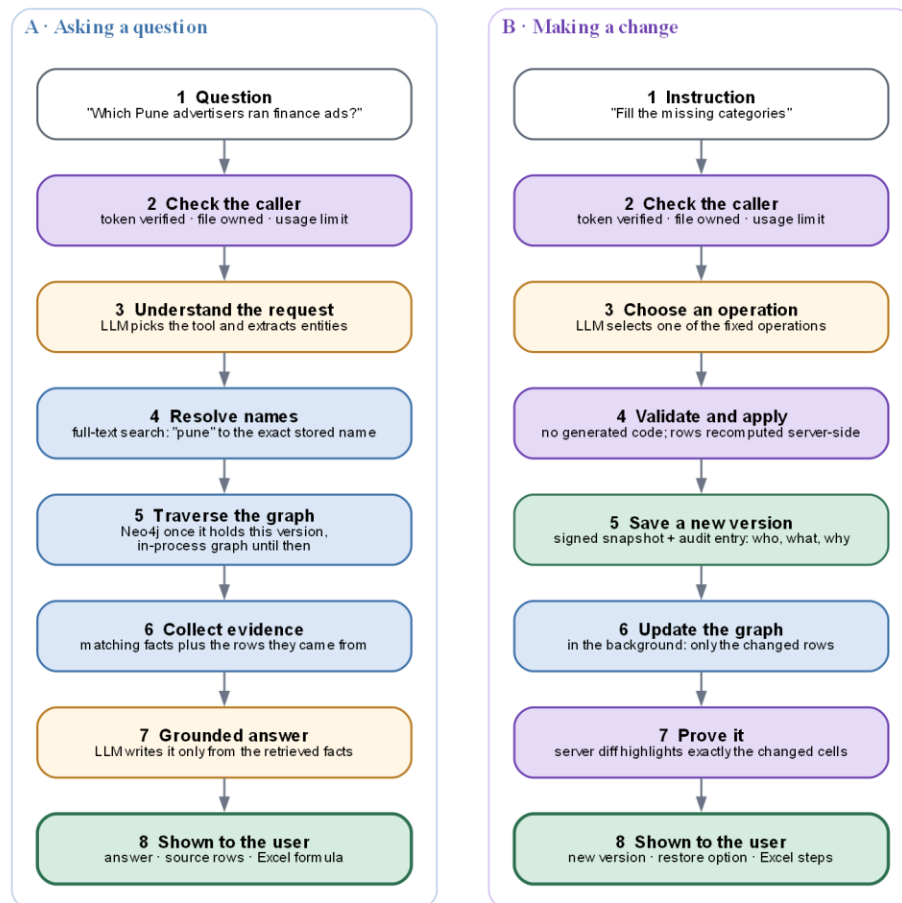


Figure 2. Question and change flows

Both flows begin with the same checks: the session token is verified, the caller must own the file, and the usage limit for assistant messages is applied. A caller over the limit receives status 429 with the time to wait.

Question flow. The assistant receives the question with the conversation history and the tool definitions. When a name may not match a stored value exactly, it first resolves the name through the full-text index. It then queries the graph, receives the matching facts with their source rows, and writes the answer from those facts only. The response includes the equivalent Excel formula.

Change flow. The assistant selects one of the fixed operations. The edit service validates the request, recomputes the affected rows on the server, and writes a new snapshot together with an audit entry recording the operation, scope, reason and user. The graph synchronisation step then updates only the affected rows in Neo4j in the background; until Neo4j holds the new version, questions are answered from the in-process graph built from the same data by the same rules. The client then requests the cell-level comparison and highlights the changed cells.

4. Technology Stack

Layer	Technology	Version	Purpose
Client	React	18.3	User interface
Client	Vite	5.4	Development server and build
Client	Clerk React SDK	6.16	Sign-in interface and session tokens
Client	lucide-react, react-markdown	1.x, 10.1	Icons and message formatting
API	Python	3.12 (hosted), 3.13 (local)	Runtime
API	FastAPI, Starlette, Uvicorn	0.141, 1.6, 0.30	HTTP service
API	pandas, openpyxl	2.2, 3.1	Workbook parsing and table operations
API	Pydantic, pydantic-settings	2.9, 2.5	Request validation and configuration
API	NetworkX	3.3	In-process graph for local mode
API	OpenAI SDK (OpenRouter)	1.57	Language model access
API	PyJWT[crypto]	2.14	Session token verification
API	psycpg, psycpg-pool	3.3	PostgreSQL driver and connection pool
API	boto3	1.43	S3-compatible snapshot storage
Data	PostgreSQL (Supabase hosted, Docker local)	17 / 16	Sessions, versions, audit log, users
Data	Neo4j (Aura hosted, Community local)	5	Knowledge graph and full-text name index
Data	Redis	7, optional	Queue for a separate graph worker
Data	Supabase Storage (S3 API) or local disk	-	Version snapshots
Identity	Clerk	-	Authentication provider
Hosting	Render (API), Vercel (client)	-	Production hosting
Tooling	Docker Compose, pytest, pip-audit, Lighthouse	-, 9.1, 2.x, 12	Local services, tests, dependency and page audits

5. Detailed Design

5.1 Schema Profiling

Each column is profiled on upload (distinct values, blank count, numeric and date detection) and assigned a role. The roles drive the diagnostics and the graph, so the system works with files it has not seen before. Users can review and change the roles in the Schema panel; a change rebuilds the graph.

Role	Rule	Used for
Subject	Repeated text column with the most distinct values; ties go to the more complete column	Graph nodes that questions are about
Attribute	Repeated values: at most 20 distinct, or at most half the row count	Graph links and grouping
Measure	Numeric column that is not unique per row	Totals and comparisons
Date	Date type, or a date-named column whose values parse as dates	Time context
Flag	At least half blank with at most three distinct values	Excluded from missing-value checks
Identifier	More than 95% distinct values	Not linked in the graph

5.2 Diagnostics

- Missing values: blank or whitespace-only cells, in all columns except flags and identifiers.
- Disagreeing columns: pairs of text columns with similar values and similar distinct-value counts that differ on some rows.
- Near-duplicates: values in the subject column with at least 90% string similarity, capped at 5,000 comparisons.

Example. In the sample file, IsHouseAd is blank in 10,595 of 11,275 rows because the source system leaves it empty for "no". It is classified as a flag and not reported as missing data.

5.3 Knowledge Graph

Each row links its subject value to its attribute values. Node identities are namespaced by column type, so identical text in different columns (for example a city and a company named after it) never becomes one node. Every relationship stores its source row index, which allows the graph to be updated per row.

Element	Neo4j representation
Node	<code>(:Node {dataset, type, name, column})</code> , unique on dataset, type and name
Relationship	<code>[:LINKED {row, column, relation, flags, props}]</code> from subject to attribute
Dataset marker	<code>(:Dataset {id, version})</code> , the last version applied
Indexes	Row index on relationships; full-text index <code>node_names</code> on node names

Synchronisation: after a change, the worker deletes the relationships produced by the changed rows, recreates them from the new data and removes nodes left without relationships. A full rebuild runs only on upload or when the column roles change.

5.4 Assistant and Tools

The assistant never writes execution code for changes. It selects a named tool and supplies typed arguments, which the dispatcher validates. A turn is limited to five tool calls.

Tool	Purpose	Changes data
<code>detect_issues</code>	Run diagnostics for the file, a column or a value	No
<code>resolve_entity</code>	Find the exact stored name for partial or misspelled input	No
<code>get_entity_summary</code>	Rows and linked values for one subject	No
<code>find_by_attributes</code>	Subjects linked to all given column values	No
<code>tabular_query</code>	Restricted table expression for counts and filters, run in an isolated process	No
<code>apply_edit</code>	Apply one of the fixed edit operations	Yes

5.5 Edit Operations

Operation	Behaviour	Safeguard
<code>fill_missing_value</code>	Fills blanks in listed rows	Refuses rows that are not blank
<code>fill_missing_value_bulk</code>	Fills every blank in a column	Recomputes the full set of rows on the server
<code>standardize_value</code>	Sets a value in listed rows	Requires an explicit, non-empty row list
<code>flag_rows</code>	Writes a flag into a review column	Never modifies source columns

Rationale. The assistant sees a row list truncated to 20 entries to limit context size. The bulk operation ignores any list it is given and recomputes the affected rows, so a request to fix all 138 missing values fixes all 138 rather than the 20 visible ones.

5.6 Versioning and Audit

- Every change stores a complete snapshot and an audit entry in one transaction.
- Restore creates a new version equal to the chosen one; later versions and their audit entries are kept.
- The pair (session, version) is a primary key. A concurrent writer that loses the race receives a conflict error instead of overwriting the other change.

5.7 Excel Formula Output

Formulas are generated from the same rules the service applies. Name comparisons use EXACT because Excel equality is case-insensitive, blank checks use TRIM to match the service definition of missing, and

cell lists for edits are taken from the actual before-and-after comparison. Addresses assume headers in row 1, which matches both the uploaded and the exported file. Formulas proposed by the language model for free-form queries are labelled as not verified.

5.8 Client Application

Screen or component	Function
Landing page	Product page with features, how it works, security and questions; sign-in and sign-up; file upload when signed in
App bar and toolbar	File name and version, data or graph view switch, assistant and export; selected cell reference and value, search, column filter, missing-only switch
Data grid	Virtualised table with column letters, sheet row numbers, sorting and search
Knowledge graph	Force-directed view of a node neighbourhood with collapsed parallel edges
Problems	Diagnostics list; selecting an issue filters the grid to its rows
History	Version table with restore and formula links
Schema	Inferred column roles with editing
Do it in Excel	Formulas and procedures for the latest answer or change
Assistant	Docked conversation panel with tool trace
Privacy policy, terms	Public legal pages at /privacy and /terms
Not-found page	Shown for any unknown address; excluded from search indexing
Cookie notice	States that only essential storage is used; dismissed once and remembered

The workspace and the secondary pages are loaded on demand, so a visitor to the landing page downloads only the code for that page. The landing page renders immediately rather than waiting for the sign-in script, and its buttons become active once that script has loaded.

5.9 Search, Sharing and Accessibility

Item	Implementation
Page titles and descriptions	Set in the page head; each route sets its own title
Link previews	Open Graph and Twitter card tags with a 1200 x 630 preview image (75 KB) and alternative text
Icons	SVG favicon, ICO fallback, touch icon and web app manifest
Sitemap and robots.txt	Generated at build time from the site address (<code>VITE_SITE_URL</code> , or the Vercel production domain)
Structured data	JSON-LD description of the application
Contrast	Body and secondary text at 4.5:1 or higher (WCAG 2.1 AA)
Keyboard and motion	Skip-to-content link, visible focus outlines, reduced motion honoured
Small screens	Layouts reflow to a single column; no horizontal scrolling at 375 px

6. Data Design

6.1 Relational Schema

The same schema is used in PostgreSQL and in SQLite (local mode). Missing columns are added automatically at startup for databases created by earlier versions.

Table	Key columns	Contents
users	user_id (PK), provider_id (unique)	Email, display name, picture, created time
sessions	session_id (PK)	File name, created time, owner_id
versions	(session_id, version) PK	Creation time for each snapshot
audit	(session_id, version) PK	Operation, scope, rows affected, reason, preview, Excel steps, user_id
conversation	(session_id, position) PK	Messages exchanged with the language model
transcript	(session_id, position) PK	Conversation as displayed in the client

6.2 Snapshot Storage

Each version is stored as a gzip-compressed pandas pickle, on local disk or in S3-compatible storage. Parquet was evaluated and rejected: the sample file contains a column mixing integers with text such as "4-5", which Parquet cannot store without converting the whole column to text and so altering the data. Snapshots are written only by the service and are never accepted from clients. Local files are written to a temporary name and renamed, so a partial write is never read as a valid version.

Because loading a pickle can execute code, every snapshot is signed. When `SNAPSHOT_SIGNING_KEY` is set, the file is prefixed with an HMAC-SHA256 signature of its contents, and a snapshot whose signature does not match is refused before it is loaded. A planted or altered file in the bucket is therefore rejected even by someone holding the storage keys. Snapshots written before signing was enabled carry no signature and are still read, so enabling it requires no migration.

Database access uses a connection pool (one to `DB_POOL_MAX` connections) instead of opening a new encrypted connection for each operation. Connections closed by the server are detected and replaced before use.

6.3 Data Ownership

When authentication is enabled, each session belongs to the user who created it. Sessions created while authentication was disabled have no owner and remain readable. Session identifiers must be canonical UUIDs before they are used to build any storage path.

7. API Specification

All endpoints return JSON unless stated otherwise. When authentication is enabled, every endpoint except `/health` and `/auth/me` requires the header `Authorization: Bearer <token>`. Interactive documentation is available at `/docs` on a running service. `/chat`, `/upload` and `/sample` are subject to usage limits (Section 8.4).

Method	Path	Description
GET	<code>/health</code>	Service status, assistant availability, authentication mode
GET	<code>/auth/me</code>	Current user, if signed in
POST	<code>/upload</code>	Upload a workbook (multipart) and start a session
POST	<code>/sample</code>	Start a session with the bundled sample file
GET	<code>/session/{id}</code>	Reopen a session: metadata, current version, transcript
GET	<code>/mapping/{id}</code>	Inferred column roles and profiles
POST	<code>/mapping/{id}</code>	Override column roles; rebuilds the graph
GET	<code>/data/{id}</code>	Paged rows with search, column filter, blank filter, row list and sort
GET	<code>/issues/{id}</code>	Diagnostics with complete row lists
GET	<code>/graph/{id}/stats</code>	Node and relationship counts
GET	<code>/graph/{id}/search</code>	Node lookup by name
GET	<code>/graph/{id}/neighborhood</code>	Subgraph around a node, parallel edges collapsed
GET	<code>/diff/{id}/{version}</code>	Every cell changed by the given version
POST	<code>/chat</code>	One assistant turn; may change data
GET	<code>/history/{id}</code>	Audit log
POST	<code>/revert</code>	Restore an earlier version as a new version
GET	<code>/export/{id}</code>	Download the current version as .xlsx

7.1 Error Responses

Status	Meaning
400	Invalid input: unsupported file, unknown column, version out of range
401	Missing, expired or invalid session token
404	Session not found or not owned by the caller
409	Version conflict; the data changed during the request
429	Usage limit reached; the <code>Retry-After</code> header and message give the time to wait
502	The language model call failed

Status	Meaning
503	A required service or credential is not configured

8. Security

8.1 Authentication

Sign-in is handled by Clerk. The client obtains a short-lived session token and sends it with each request. The service verifies the RS256 signature against Clerk's published keys, which are fetched once and cached, and checks expiry, issuer and the authorised origin (azp). The Clerk domain and key URL are derived from the publishable key.

8.2 Authorisation

- A signed-in user can access only the sessions they created.
- Requests for another user's session return 404, so the existence of a session is not disclosed.
- Each audit entry records the acting user.

8.3 Other Controls

- The write path executes no generated code.
- Session identifiers are validated as UUIDs before use in storage paths.
- Secrets are supplied through environment variables; `.env` files are excluded from version control.
- Cross-origin requests are limited to the configured client origins.
- Snapshots are signed and verified before loading (Section 6.2).
- The client contains no secrets; its only key is the Clerk publishable key, which is public by design.

8.4 Usage Limits

Limits apply per signed-in user, or per client address when sign-in is disabled (the first address in `X-Forwarded-For` behind the hosting proxy). Accounts listed in `RATE_LIMIT_EXEMPT`, by Clerk user ID or email, are never limited, so a demonstration cannot be interrupted.

Action	Default limit	Endpoints
Assistant messages	30 per hour and 100 per day	/chat
File loads	10 per hour	/upload, /sample

A request over the limit returns 429 with a `Retry-After` header and a message such as "Please try again in 12 minutes", which the client shows to the user.

8.5 Query Isolation

Table queries written by the language model are validated against a list of forbidden constructs and then evaluated in a separate process, never in the API process.

- Wall-clock limit of 10 seconds; the process is terminated if it does not answer.
- CPU limit of 5 seconds on Linux.
- Network access disabled inside the process.

- Results capped at 500 rows; only tables, series or values are returned.
- Processes start from a clean single-threaded server on Linux (forkserver) and by spawning on Windows.

8.6 Transport and Browser Security

Header	Value	Applied by
Strict-Transport-Security	two years, sub-domains, preload	Client host, API
Content-Security-Policy	upgrade-insecure-requests; frame-ancestors none (client); default-src none (API)	Client host, API
X-Frame-Options	DENY	Client host, API
X-Content-Type-Options	nosniff	Client host, API
Referrer-Policy	strict-origin-when-cross-origin	Client host, API
Permissions-Policy	camera, microphone, geolocation disabled	Client host, API

Both hosts redirect plain HTTP to HTTPS. Connections to PostgreSQL, storage and Neo4j are encrypted (TLS).

8.7 Dependency Audit

Python dependencies are checked with pip-audit and client dependencies with npm audit. The v1.1 audit found 30 advisories in FastAPI/Starlette, python-multipart and pytest; these were upgraded (FastAPI 0.141.1, Starlette 1.6.0, python-multipart 0.0.32, pytest 9.1.1) and both audits now report no known vulnerabilities.

8.8 Residual Risks

Item	Risk	Mitigation
Credentials shared outside the secret store	Database, storage or graph passwords may be exposed	Rotate them and update the hosting environment
Usage limit counters	Kept per API process and reset on restart	Move counters to Redis when running several instances
Language model key	Public access can consume paid credits	Usage limits; provider spending limit
Demo mode	Authentication disabled	Use only for demonstrations with sample data

9. Configuration

Backend settings are read from environment variables or `backend/.env`. Environment variables take precedence.

Variable	Default	Description
OPENROUTER_API_KEY	empty	Language model key; required for the assistant
OPENROUTER_MODEL	openrouter/auto	Model identifier
DATABASE_URL	empty	PostgreSQL connection string; empty selects SQLite
NEO4J_URI, NEO4J_USER, NEO4J_PASSWORD	empty, neo4j, empty	Neo4j connection; empty URI selects the in-process graph
NEO4J_DATABASE	empty	Database name; Aura uses the instance ID
REDIS_URL	empty	Queue for graph updates; empty applies updates inline
S3_ENDPOINT_URL, S3_ACCESS_KEY, S3_SECRET_KEY, S3_BUCKET	empty	Snapshot storage; empty selects local disk
S3_REGION	us-east-1	Storage region; Supabase requires the project region
SNAPSHOT_SIGNING_KEY	empty	HMAC key for snapshots; long random string
DB_POOL_MAX	5	Maximum pooled database connections
DATA_DIR	data	Local data directory
AUTH_ENABLED	false	Require sign-in
CLERK_PUBLISHABLE_KEY	empty	Clerk key used to locate the signing keys
CORS_ALLOW_ORIGINS	localhost:5173	Allowed client origins (JSON list)
MAX_UPLOAD_SIZE_MB	25	Upload size limit
RATE_LIMIT_ENABLED	true	Apply usage limits
RATE_LIMIT_CHAT_PER_HOUR, RATE_LIMIT_CHAT_PER_DAY	30, 100	Assistant message limits
RATE_LIMIT_UPLOADS_PER_HOUR	10	File load limit
RATE_LIMIT_EXEMPT	empty	Comma-separated Clerk user IDs or emails never limited

Client settings are read from `frontend/.env` at build time.

Variable	Description
VITE_API_BASE	Base URL of the API service
VITE_CLERK_PUBLISHABLE_KEY	Clerk publishable key; empty runs without sign-in

Variable	Description
VITE_SITE_URL	Public site address for canonical links, previews and the sitemap; defaults to the Vercel production domain
VITE_CONTACT_EMAIL	Contact address shown in the footer and legal pages

10. Deployment

10.1 Environments

Mode	Backing services	Use
Local	SQLite, local files, in-process graph	Development without Docker
Local with services	PostgreSQL, Neo4j, Redis in Docker Compose	Development and integration testing
Production	Supabase PostgreSQL and Storage, Neo4j Aura; Redis not required	Hosted deployment

10.2 Local Setup

1. Install Python 3.10 or later, Node.js 18 or later and Docker Desktop.
2. Create the Python environment and install dependencies:

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
```

3. Install client dependencies:

```
cd frontend
npm install
```

4. Start the backing services from the project root:

```
docker compose up -d
```

5. Start the API, the worker and the client in separate terminals:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000      (backend)
python -m app.graph_worker                          (backend)
npm run dev                                           (frontend)
```

6. Open <http://localhost:5173>.

10.3 Production Deployment

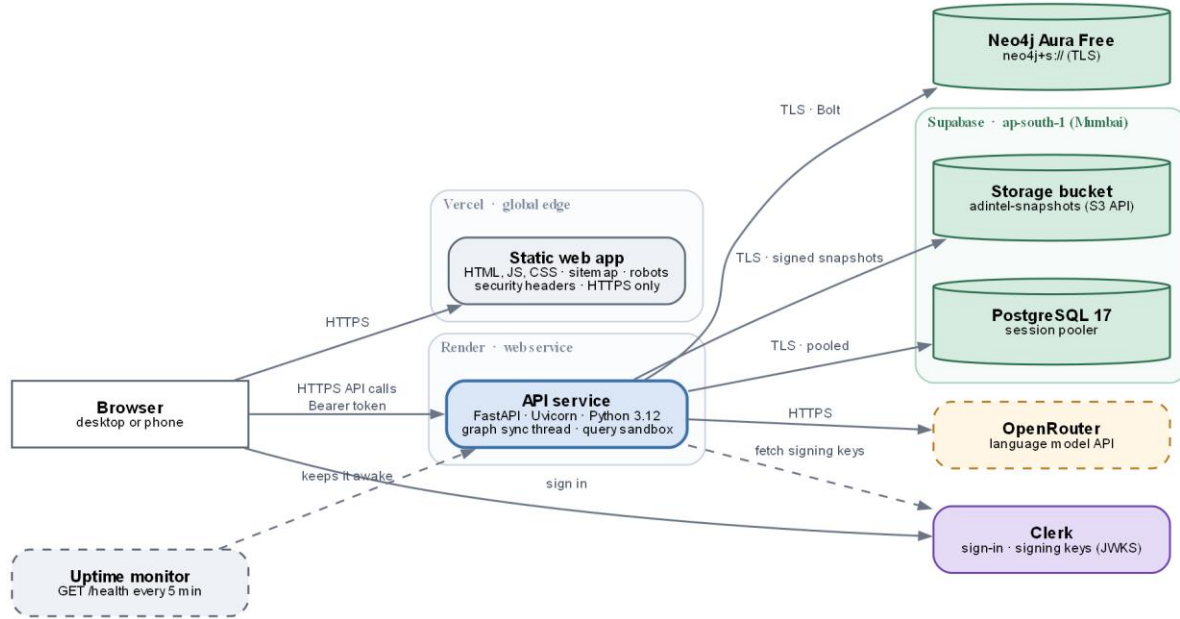


Figure 3. Hosted deployment

Component	Platform	Notes
API	Render web service	Root backend; start command <code>uvicorn app.main:app --host 0.0.0.0 --port \$PORT; PYTHON_VERSION=3.12.7</code>
Client	Vercel	Root directory <code>frontend</code> , Vite preset; headers and routing in <code>vercel.json</code>
PostgreSQL	Supabase (ap-south-1)	Session pooler connection string; characters such as <code>@</code> in the password must be URL-encoded (<code>%40</code>)
Snapshots	Supabase Storage	Bucket <code>adintel-snapshots</code> through the S3 API; <code>S3_REGION=ap-south-1</code>
Neo4j	Neo4j Aura Free	URI <code>neo4j+s://...</code> ; <code>NEO4J_DATABASE</code> set to the instance ID
Identity	Clerk	Client address added to <code>CORS_ALLOW_ORIGINS</code> on the API
Monitoring	Uptime monitor	Requests <code>/health</code> every 5 minutes so the free instance does not sleep

10.4 Release Procedure

1. Commit and push to the `main` branch; Render and Vercel build and deploy automatically.
2. Set new or changed variables in each platform's environment settings, never in the repository.
3. After the first client deployment, set `CORS_ALLOW_ORIGINS` on the API to the client address.
4. Check `GET /health` on the API and open the client address.

10.5 Service Endpoints (Local)

Service	Address	Credentials
Client	http://localhost:5173	Clerk account
API and documentation	http://localhost:8000/docs	-
PostgreSQL viewer (Adminer)	http://localhost:8080	Server postgres, user/password/database adintel
Neo4j Browser	http://localhost:7474	neo4j / adintel-graph

11. Performance

11.1 Server

Measure	Effect
Connection pooling	Database operations reuse open encrypted connections instead of opening one per operation
Background graph sync	Uploads and edits return without waiting for Neo4j; answers use the in-process graph until Neo4j catches up
Uptime monitor	Prevents the free hosting instance from sleeping, avoiding a cold start of up to a minute
Region placement	Placing the API in the same region as the database (Mumbai / Singapore) removes cross-continent round trips

11.2 Client

- Code splitting: the workspace, legal pages and not-found page load on demand; vendor code is split into long-lived chunks.
- Hashed assets are served with a one-year immutable cache.
- Fonts are preconnected and loaded from the page head rather than imported inside a stylesheet.
- The landing page does not wait for the sign-in script before rendering.

11.3 Measured Results

Lighthouse 12 against the production build of the landing page, using its default mobile profile (slow 4G, mid-range device):

Category	Score	Notes
Performance	78	First contentful paint 1.7 s; largest contentful paint 2.8 s (4.0 s before the changes above)
Accessibility	100	Contrast, labels, landmarks and focus order
Best practices	79	Lowered by cookies set by the Clerk development instance on its own domain
SEO	100	Titles, descriptions, crawlable links, robots.txt

The remaining blocking time comes mainly from the sign-in library. A Clerk production instance on the application's own domain removes the third-party cookies.

12. Testing

12.1 Strategy

- Unit and API tests run without network services or an API key. The test configuration isolates storage and disables external services.
- An integration script verifies the PostgreSQL, Neo4j and synchronisation path against the full sample file.
- Several tests reproduce defects found during development so that they cannot recur.

12.2 Test Inventory

Suite	Tests	Coverage
<code>test_security.py</code>	11	Usage limits and exemptions, 429 responses, security headers, snapshot signing and tamper detection, query isolation and timeout
<code>test_ide_endpoints.py</code>	24	Read endpoints, strict JSON, search, sorting, comparison, restore, formula output
<code>test_auth.py</code>	10	Ownership, audit actor, token signature, expiry, issuer and origin checks
<code>test_persistence.py</code>	9	Restart recovery, exact value preservation, restore, version conflicts, path validation
<code>test_agent_dispatch.py</code>	8	Tool dispatch without a language model
<code>test_safe_editor.py</code>	8	Edit operations and their refusals
<code>test_graph_builder.py</code>	5	Graph construction and node identity
<code>test_diagnostics.py</code>	4	Issue detection and flag handling
<code>integration_neo4j.py</code>	12 checks	PostgreSQL storage, Neo4j parity with the in-process graph, row sync, name resolution

12.3 Results

Unit and API tests: 79 passed. Integration checks: 12 passed. On the sample file, Neo4j and the in-process graph hold the same 10,698 nodes and 112,010 relationships and return identical answers.

```
cd backend
pytest tests/ -q
python tests/integration_neo4j.py
```

13. Operations and Maintenance

13.1 Health Checks

Check	Command or location	Expected result
Containers	<code>docker compose ps</code>	All services running
API	<code>GET /health</code>	"status": "ok"
Hosted API	<code>https://excelai-vnua.onrender.com/health</code>	"status": "ok", "auth_enabled": true
Worker	Worker console	"Graph worker listening"
PostgreSQL	Adminer, <code>sessions</code> table	A row per uploaded file
Neo4j	<code>MATCH (n:Node) RETURN count(n)</code>	Nodes present after an upload

13.2 Troubleshooting

Symptom	Cause	Resolution
Browser reports a CORS error	API bound only to 127.0.0.1 while the client uses localhost	Start the API with <code>--host 0.0.0.0</code>
All containers exit together	Docker Desktop stopped, often due to low disk space	Free disk space or move the Docker disk image; restart Docker
Neo4j exits on startup	Invalid or corrupted configuration	Check <code>docker compose logs neo4j</code> ; recreate the container
Worker cannot reach Neo4j	Neo4j still starting	Wait; the worker retries every 5 seconds
Requests return 401	Missing or expired token, or mismatched Clerk key	Sign in again; check both publishable keys match
Port already in use	A previous process is still running	Stop the process holding port 8000 or 5173
First request takes up to a minute	Free hosting instance was asleep	Configure the uptime monitor
Requests return 429	Usage limit reached	Wait for the time shown, or add the account to <code>RATE_LIMIT_EXEMPT</code>
Snapshot refused as tampered	<code>SNAPSHOT_SIGNING_KEY</code> changed or removed	Restore the original key; it must never change once set
Database connection fails at startup	Special characters in the password not encoded	URL-encode the password in <code>DATABASE_URL</code>

13.3 Extending the System

- New diagnostic: add a read-only function in `diagnostics.py` and include it in the full check.
- New edit operation: add a function in `safe_editor.py`, register it, add it to the tool definition and add tests for its refusals.

- New column role rule: update `schema_profile.py`; the graph and diagnostics follow automatically.
- Changes to data model or security behaviour should be recorded in `docs/DECISIONS.md`.

14. Known Limitations and Future Work

Area	Limitation	Planned improvement
Storage	Each version is a full copy of the table	Store differences between versions
Usage limits	Counters are per process and reset on restart	Shared counters in Redis
Hosting	Free instance sleeps without traffic and has limited memory	Paid instance in the database region
Client performance	Sign-in library adds blocking time on slow phones	Clerk production instance; defer loading until needed
Legal pages	Written as a template	Review by a qualified adviser before commercial use
Near-duplicate check	Comparison count capped at 5,000	Blocking or approximate matching
Graph panel	Visualisation reads the in-process graph	Serve the panel from Neo4j
Formula output	Formulas not executed in Excel automatically	Automated check through Excel automation
Workbooks	Only the first worksheet is read	Worksheet selection on upload

Appendix A. Repository Structure

```
ad-intel-agent/  
  backend/  
    app/          agent, auth, config, data_store, diagnostics, diff,  
                  excel_formulas, graph_backend, graph_builder, graph_cache,  
                  graph_sync, graph_worker, main, neo4j_graph, safe_editor,  
                  safe_executor, isolated_exec, rate_limit, schema_profile,  
                  schemas, snapshot_storage  
    tests/        unit, API, authentication and integration tests  
    sample_data/  Sample.xlsx  
  frontend/  
    src/          App, api, main, site, styles (workspace, landing)  
    src/components AgentPanel, CookieNotice, DataGrid, FormulaPanel, GraphPanel,  
                  HistoryPanel, LandingPage, LegalPage, NotFound, ProblemsPanel,  
                  SchemaPanel, SiteFooter, Splitter  
    public/       favicon, touch icons, manifest, social preview image  
    vercel.json   routing, security headers, caching  
  docs/          architecture, decisions, data model, API, maintenance, diagrams  
  docker-compose.yml  
  RUN.md
```

Appendix B. Sample Dataset

Property	Value
Rows and columns	11,275 rows, 16 columns
Content	Newspaper advertisement insertions: advertiser, publication, category, location, sales office, size
Inferred subject	Advertiser Name by AI
Inferred flag	IsHouseAd
Issues found	6 columns with missing values (138 in Category); 904 name disagreements across 1,574 rows
Graph	10,698 nodes, 112,010 relationships